Day 3 - Object-Oriented Programming

Prepared by: Sarthak Jejurkar

1. OOP Concepts

Concept	Simple meaning	Example
Class and object	A class is a blueprint; an object is a copy made from it.	`s = Student()`
Constructor	Runs automatically when an object is created.	`def __init__(self)`
Encapsulation	Keeps data private and controls access through methods.	`self.__marks`
Inheritance	A child class reuses the parent class.	`class Student(Person)`
Polymorphism	Same method name, different behaviour.	`p.introduce()`
Abstraction	Hides details; forces subclasses to define a method.	`@abstractmethod`
Access conventions	public: *name*, protected: *_name*, private: *__name*.	`self._age`
Method overriding	Child class redefines a parent method.	`def introduce(self)`

2. Program 1: Student and Employee Classes

Demonstrates constructors, encapsulation, inheritance, polymorphism, abstraction and overriding.

```
from abc import ABC, abstractmethod


class Person(ABC):                      # Abstraction
    def __init__(self, name, age):      # Constructor
        self.name = name                # public
        self._age = age                 # protected

    @abstractmethod
    def role(self):
        pass

    def introduce(self):
        return f"I am {self.name}, {self._age} years old."


class Student(Person):                  # Inheritance
```

```
    def __init__(self, name, age, marks):
        super().__init__(name, age)
        self.__marks = marks            # private (Encapsulation)

    def get_marks(self):
        return self.__marks

    def set_marks(self, marks):
        if 0 <= marks <= 100:
            self.__marks = marks
        else:
            print("Invalid marks!")

    def role(self):
        return "Student"

    def introduce(self):                # Method overriding
        return super().introduce() + f" Marks: {self.__marks}"


class Employee(Person):
    def __init__(self, name, age, salary):
        super().__init__(name, age)
        self.__salary = salary

    def role(self):
        return "Employee"

    def introduce(self):
        return super().introduce() + f" Salary: {self.__salary}"


s = Student("Aarav", 20, 88)
e = Employee("Rohan", 30, 50000)

s.set_marks(150)                        # rejected by validation
s.set_marks(95)

for p in [s, e]:                        # Polymorphism
    print(p.role(), "->", p.introduce())

try:
    print(s.__marks)
except AttributeError:
    print("Private attribute cannot be accessed directly")
```

Output

```
Invalid marks!
Student -> I am Aarav, 20 years old. Marks: 95
Employee -> I am Rohan, 30 years old. Salary: 50000
Private attribute cannot be accessed directly
```

3. Program 2: Student Management Console App

A menu-driven app to add, view, search and delete students, built with a **Student** class and a **StudentManager** class.

```
class Student:
    def __init__(self, roll, name, marks):
        self.roll = roll
        self.name = name
        self.__marks = marks            # private

    def grade(self):
        if self.__marks >= 80:
            return "A"
        if self.__marks >= 60:
            return "B"
        return "C"

    def show(self):
        print(f"{self.roll:<6}{self.name:<12}{self.__marks:<7}{self.grade()}")


class StudentManager:
    def __init__(self):
        self.students = []

    def add(self, student):
        self.students.append(student)
        print("Student added.")

    def view(self):
        if not self.students:
            print("No students yet.")
            return
        print("Roll  Name        Marks  Grade")
        for s in self.students:
            s.show()

    def search(self, roll):
        for s in self.students:
            if s.roll == roll:
                s.show()
                return
        print("Student not found.")

    def delete(self, roll):
        for s in self.students:
            if s.roll == roll:
                self.students.remove(s)
                print("Student deleted.")
                return
        print("Student not found.")


manager = StudentManager()
while True:
    print("\n1.Add  2.View  3.Search  4.Delete  5.Exit")
    choice = input("Enter choice: ")
    try:
        if choice == "1":
            roll = int(input("Roll no: "))
            name = input("Name: ")
            marks = int(input("Marks: "))
```

```
            manager.add(Student(roll, name, marks))
        elif choice == "2":
            manager.view()
        elif choice == "3":
            manager.search(int(input("Roll no: ")))
        elif choice == "4":
            manager.delete(int(input("Roll no: ")))
        elif choice == "5":
            print("Goodbye!")
            break
        else:
            print("Invalid choice.")
    except ValueError:
        print("Please enter numbers only.")
```

Sample run

```
1.Add  2.View  3.Search  4.Delete  5.Exit
Enter choice: 1
Roll no: 1
Name: Aarav
Marks: 88
Student added.

1.Add  2.View  3.Search  4.Delete  5.Exit
Enter choice: 1
Roll no: 2
Name: Diya
Marks: 93
Student added.

1.Add  2.View  3.Search  4.Delete  5.Exit
Enter choice: 1
Roll no: 3
Name: Kabir
Marks: 55
Student added.

1.Add  2.View  3.Search  4.Delete  5.Exit
Enter choice: 2
Roll  Name        Marks  Grade
1     Aarav       88     A
2     Diya        93     A
3     Kabir       55     C

1.Add  2.View  3.Search  4.Delete  5.Exit
Enter choice: 3
Roll no: 2
2     Diya        93     A

1.Add  2.View  3.Search  4.Delete  5.Exit
Enter choice: 4
Roll no: 1
Student deleted.

1.Add  2.View  3.Search  4.Delete  5.Exit
Enter choice: 2
Roll  Name        Marks  Grade
2     Diya        93     A
3     Kabir       55     C

1.Add  2.View  3.Search  4.Delete  5.Exit
Enter choice: 5
Goodbye!
```